

ProtoLens su LexGLUE Unfair-ToS

Simone Cappugi

E-mail address

simone.cappugi@edu.unifi.it

Abstract

L'analisi dei Termini di Servizio (ToS) richiede l'utilizzo di modelli in grado non solo di individuare accuratamente le clausole unfair, ma anche di fornire spiegazioni interpretabili a supporto delle decisioni. In questa relazione si presenta un'implementazione estesa e ottimizzata di ProtoLens, un'architettura IA interpretabile basata su prototipi (prototype-based), applicata al dataset legale **LexGLUE Unfair-ToS**. Il modello integra un estrattore di feature basato su reti transformer (Sentence-BERT) con un modulo di attenzione alle parti (MDN - Mixture Density Network) e un meccanismo di masking adattivo (Adaptive Masking). Tali componenti permettono al modello di isolare e classificare i frammenti di testo rilevanti confrontandoli con prototipi appresi durante l'addestramento.

Al fine di mitigare lo sbilanciamento del dataset, l'implementazione originale è stata modificata mediante tecniche di oversampling (WeightedRandomSampler) e, soprattutto, attraverso un'inizializzazione supervisionata dei prototipi, applicando algoritmi di clustering (K-Means) separatamente su ciascuna classe.

La sperimentazione è stata strutturata in tre fasi analitiche:

- caso "OR" (binario): task volto a determinare la presenza o assenza di almeno una tra le 8 tipologie di clausole unfair in un determinato testo.
- **Caso Multiclass:** Un' estensione dell' architettura per la classificazione esatta della specifica categoria di unfairness.
- **Caso Monoclass:** Un approccio basato sull'addestramento di tre modelli distinti e specializzati sulle tre classi unfair più popolose..

Le prestazioni delle tre configurazioni sono state valutate combinando metriche di classificazione classiche (Accuracy, F1-Score) con metriche di Explanation Quality (Top-K Match Rate), le quali misurano la coerenza semantica tra le categorie dei prototipi attivati e la classificazione reale.

1. Introduction

ProtoLens è un'architettura di reti neurali basata su prototipi (prototype-based). A differenza dei classici modelli "black-box", ProtoLens non si limita a classificare un testo, ma ne evidenzia i frammenti più rilevanti e li confronta con i "prototipi" appresi durante il training, fornendo all'utente una giustificazione (explainability).

Il seguente lavoro descrive l'implementazione, di ProtoLens sul dataset LexGLUE Unfair-ToS. Questo dataset è strutturato in 8 diverse categorie di clausole vessatorie. Come emerso dall'analisi esplorativa dei dati, il dataset presenta un forte sbilanciamento delle classi fig:1. Tale sbilanciamento rappresenta una sfida significativa per l'addestramento dei modelli.

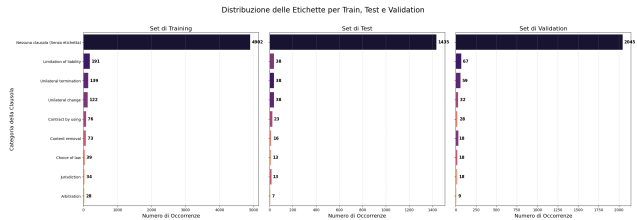


Figure 1. Distribuzione del dataset

1.1. Costruzione e utilizzo dei Prototipi

Per la costruzione dei prototipi è stato implementato lo script `build_prototypesV2.py`, che introduce una procedura di inizializzazione supervisionata rispetto alle 8 categorie originali del dataset LexGLUE Unfair-ToS.

A differenza di un clustering globale sull'intero dataset, la procedura costruisce separatamente i prototipi associati a ciascuna categoria. Gli step principali sono i seguenti:

- **Selezione delle clausole per categoria:** il processo itera sulle 8 categorie disponibili (da 0 a 7) e, per ciascuna di esse, seleziona dal file `train_multilabel.csv` esclusivamente le clausole etichettate positivamente per la categoria considerata.

- **Calcolo degli embeddings:** le clausole selezionate vengono convertite in rappresentazioni vettoriali dense mediante il modello *sentence-transformers/all-mpnet-base-v2*, utilizzato anche nell'architettura ProtoLens impiegata negli esperimenti.
- **Clustering K-Means per categoria:** sugli embeddings appartenenti alla singola categoria viene applicato separatamente l'algoritmo K-Means. Il numero di cluster K assegnato a ciascuna categoria è determinato dal parametro *prototypes_per_class*. Di conseguenza, il numero complessivo di prototipi è dato da:

$$N_{\text{prototipi}} = N_{\text{categorie}} \times \text{prototypes_per_class}.$$

Nel caso di Unfair-ToS, essendo presenti 8 categorie, con *prototypes_per_class*=3 si ottengono complessivamente 24 prototipi.

- **Selezione delle clausole rappresentative:** per ciascun cluster, il centroide prodotto da K-Means viene utilizzato per inizializzare il corrispondente prototipo. Successivamente, tramite similarità coseno, vengono selezionate le 15 clausole del cluster più vicine al centroide, che costituiscono un *pool* di esempi rappresentativi associato al prototipo.

Tale pool non viene utilizzato esclusivamente a fini interpretativi, ma interviene anche durante l'addestramento. Periodicamente (ogni 50 batch), il modello confronta il vettore corrente del prototipo con gli embeddings delle clausole del relativo pool, seleziona le tre più simili e ne calcola una rappresentazione media. Quest'ultima viene utilizzata per riallineare il prototipo verso esempi testuali reali e semanticamente coerenti con il cluster di origine, contribuendo a preservarne l'interpretabilità durante l'ottimizzazione.

La procedura genera tre file utilizzati nelle successive fasi di addestramento e interpretazione:

- *<dataset>_cluster_<K>_centers.npy*, contenente i vettori dei centroidi utilizzati per inizializzare i prototipi;
- *<dataset>_cluster_<K>_to_sub_sentence.csv*, contenente, per ciascun prototipo, il pool delle clausole reali più rappresentative;
- *<dataset>_prototype_metadata.json*, che associa a ciascun prototipo la categoria di origine, il cluster di appartenenza e una clausola rappresentativa. Quest'ultimo file viene utilizzato successivamente per ricondurre l'attivazione dei prototipi alle categorie originali e valutare la qualità delle spiegazioni prodotte dal modello.

1.1.1 Forward Pass del Modello

La procedura descritta di seguito rappresenta il *forward pass* dell'architettura ProtoLens ed è condivisa tra la fase di addestramento e quella di inferenza. Durante il training, a tale procedura si aggiungono il calcolo della funzione di costo, l'aggiornamento dei parametri e il periodico riallineamento dei prototipi rispetto alle clausole rappresentative associate a ciascun prototipo.

Il confronto tra testo e prototipi non viene effettuato direttamente sull'intero documento. L'input viene infatti trasformato in un insieme di frammenti locali mediante un *CountVectorizer* configurato con

```
ngram_range = (window_size, window_size).
```

In questo modo vengono costruiti n-grammi composti da un numero di parole pari al parametro *window_size*. Tali frammenti costituiscono le unità candidate utilizzate per individuare le porzioni del testo semanticamente più affini ai prototipi.

Il forward pass può essere suddiviso nelle seguenti fasi:

- **1. Matching tra frammenti e prototipi:** il metodo *get_start_point* estrae gli n-grammi presenti nell'input e ne calcola gli embeddings mediante il modello *SentenceTransformer*.

Gli embeddings dei frammenti vengono quindi confrontati con i vettori dei prototipi tramite similarità coseno. Per ogni esempio viene così ottenuto il tensore *chunk_similarity*, che descrive il grado di affinità semantica tra ciascun frammento candidato e ciascun prototipo.

- **2. Elaborazione tramite Mixture Density Network:** i valori di similarità vengono forniti a una *Mixture Density Network* (MDN), che restituisce i parametri π , μ e σ di una miscela di componenti.

Tali parametri vengono utilizzati per modellare la distribuzione delle regioni candidate considerate rilevanti rispetto ai prototipi, consentendo al modello di superare una selezione basata esclusivamente sugli n-grammi iniziali di lunghezza fissa.

- **3. Mascheramento adattivo:** i parametri prodotti dalla MDN vengono passati al modulo *AdaptiveMask*, che genera una maschera binaria utilizzata per selezionare gli elementi della rappresentazione dell'input maggiormente rilevanti.

In particolare, il modulo utilizza le componenti ritenute più significative sulla base dei valori di π e costruisce una maschera adattiva a partire dai parametri μ e σ .

- **4. Costruzione della rappresentazione locale:** il backbone transformer produce gli embeddings contestuali dei token dell'input. Per ciascun prototipo p , la maschera generata dal modulo `AdaptiveMask` seleziona le porzioni del testo considerate rilevanti rispetto a tale prototipo.

Gli embeddings dei token selezionati vengono quindi aggregati mediante *mean pooling*, ottenendo una rappresentazione locale Z'_p . Tale vettore non rappresenta l'intero documento, ma sintetizza la porzione dell'input che il modello considera maggiormente informativa rispetto al prototipo p .

- **5. Attivazione dei prototipi e classificazione:** per ciascun prototipo p , viene calcolata la similarità coseno tra la rappresentazione locale Z'_p e il corrispondente vettore prototipale P_p :

$$s_p = \cos(Z'_p, P_p).$$

Il valore s_p può essere interpretato come il grado di attivazione del prototipo p sull'input corrente. Le attivazioni di tutti i prototipi vengono raccolte in un vettore

$$\mathbf{s} = [s_1, s_2, \dots, s_P],$$

che viene infine fornito al livello completamente connesso `self.fc` per ottenere i logiti e la predizione finale.

1.2. Mitigazione dello Sbilanciamento

Come evidenziato dall'analisi preliminare, il dataset LexGLUE Unfair-ToS soffre di un severo sbilanciamento delle classi. Se non gestito, tale squilibrio indurrebbe il modello a favorire la classe maggioritaria (clausole "fair"), non riuscendo quasi mai ad individuare le clausole unfair.

Per mitigare questo problema, oltre all'inizializzazione supervisionata dei prototipi descritta in precedenza, è stata introdotta una tecnica di oversampling nel *DataLoader*. Nello specifico, è stato implementato un campionamento pesato tramite `WeightedRandomSampler` di PyTorch. Per ogni istanza del set di addestramento è stato calcolato un peso inversamente proporzionale alla frequenza della sua classe di appartenenza ($Peso = 1.0 / Frequenza_Classe$). Il campionatore estrae i batch di training basandosi su tali probabilità, garantendo che il modello "veda" un numero bilanciato di esempi positivi e negativi durante l'apprendimento, questo stabilizza l'aggiornamento dei pesi e la generazione della loss.

1.3. Gli Scenari di Classificazione

Per valutare la robustezza e la versatilità dell'architettura ProtoLens, la sperimentazione è stata suddivisa in tre diversi scenari analitici:

1. **Caso "OR" (Classificazione Binaria):** In questo primo scenario, il task multi-etichetta originale è stato trattato come un problema binario. Un documento viene etichettato come positivo ($Label = 1$) se contiene **almeno una** clausola appartenente a una qualsiasi delle 8 categorie *unfair*, e negativo ($Label = 0$) altrimenti.
2. **Caso Multiclasse:** In questo scenario, l'architettura è stata estesa per eseguire una classificazione granulare, ponendosi l'obiettivo di individuare non solo la natura vessatoria del documento, ma anche la specifica categoria legale violata (es. *Limitation of Liability*, *Unilateral Change*, ecc.).
3. **Modelli Binari Specializzati:** A causa dell'estrema scarsità di campioni per alcune categorie minoritarie, l'addestramento di un unico modello multiclasse può risultare non ottimale. Pertanto, si è optato per l'addestramento di tre modelli binari di esempio indipendenti, ciascuno specializzato nel riconoscimento di una delle tre classi *unfair* più popolose del dataset. Questo approccio divide il problema complesso in task più semplici, lo scopo sarebbe quello di massimizzare la precisione sulle clausole più frequenti.

2. Analisi dei Risultati (Caso OR)

Andiamo ora a analizzare il primo modello e come varia al variare di alcuni iperparametri. Per valutare l'impatto degli iperparametri sulle performance predittive e sulla qualità dell'interpretabilità, è stata condotta una *Grid Search* testando tre diversi valori di *Learning Rate* (0.0001, 0.0005 e 0.001) e due diverse dimensioni della finestra di scansione testuale, ovvero la *Window Size* (3 e 5). In questi esperimenti il numero di prototipi è stato fissato a 3 (24 prototipi totali).

2.1. Meccanismo di estrazione del Top-K nel Task Binario

Nel contesto del Caso "OR", abbiamo una sfida architetturale legata all'interpretabilità: il modello viene addestrato esclusivamente su etichette binarie (0 per *Fair*, 1 per *Unfair*) perdendo formalmente la granularità delle 8 categorie legali durante il calcolo della loss. Tuttavia, il sistema riesce comunque a valutare la *Explanation Quality* grazie a una validazione tra i metadati dei prototipi e la *ground truth* multi-etichetta.

Il processo di calcolo del **Top-K Category Match Rate** si articola nei seguenti passaggi:

1. **Tracciamento dei Prototipi:** Durante la fase di clustering iniziale, lo script `build_prototypesV2.py` genera un file (`prototype_metadata.json`) che funge da "memoria". Questo file mappa l'identificativo di ciascun prototipo addestrabile alla sua categoria originale (da 0 a 7) e alla frase reale da cui è stato generato.
2. **Calcolo del Contributo (Inferenza):** Durante l'inferenza, il metodo `explain_prediction` definito in `PLens.py` stima il contributo di ciascun prototipo alla decisione finale del classificatore.

Per una classe predetta $\hat{y}$, il contributo del prototipo p viene calcolato come:

$$C_p = s_p \cdot w_{\hat{y},p},$$

dove s_p rappresenta la similarità tra la rappresentazione del testo in input e il prototipo p , mentre $w_{\hat{y},p}$ rappresenta il peso associato al prototipo p nel livello di classificazione finale per la classe predetta $\hat{y}$.

I prototipi vengono quindi ordinati in ordine decrescente rispetto al valore di C_p , ottenendo così i prototipi che hanno fornito il contributo maggiore alla predizione. In questo modo è possibile estrarre i primi K prototipi più influenti, ad esempio Top-1, Top-2 e Top-3.

Grazie ai metadati memorizzati nel file JSON, per ciascun prototipo selezionato è inoltre possibile recuperare la categoria associata e la relativa frase rappresentativa iniziale.

3. **Match Rate Evaluation:** La valutazione effettiva avviene incrociando questi dati con il file `test_multilabel.csv`, tenuto nascosto durante l'addestramento ma utilizzato dal metodo `evaluate_explanation_quality` per fini di XAI. Per ogni documento correttamente predetto come *Unfair* dal modello binario, il sistema verifica se la categoria del prototipo attivato sia effettivamente presente tra le etichette reali di quel documento (es. `label_1` e `label_7`).

L'obiettivo di questo scenario sarebbe quello di avere un ottimo match rate così da dimostrare che il modello OR sia addestrato per emettere un semplice allarme binario, internamente continua a rispettare le 8 categorie unfair, attivando il prototipo "giusto" per la ragione "giusta".

2.2. Analisi Individuale dei Modelli

Modello 1: Learning Rate = 0.0001, Window Size = 3

Dal grafico delle curve di apprendimento (fig:2), si evince che il modello converge in modo graduale e stabile, raggiungendo un plateau delle metriche di validazione intorno all'epoca 15. Dal punto di vista predittivo, il modello ottiene un'accuratezza globale dello 0.755 e un F1-Score sulla classe *unfair* pari a 0.412. Risulta particolarmente notevole l'altissima Recall (0.848), indicando che il modello individua quasi tutte le clausole unfair, a discapito di una Precision contenuta (0.272). L'interpretabilità è molto buona: il *Top-1 Category Match Rate* è del 40.7%, e sale al 60.7% se si considerano i primi tre prototipi (Top-3).

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.978	0.745	0.845	2045
unfair	0.272	0.848	0.412	230
Accuracy			0.755	2275
Macro Avg	0.625	0.796	0.629	2275
Weighted Avg	0.906	0.755	0.802	2275

Table 1. Classification Report per il modello LR_0.0001.Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	150
Top-1 Category Match Rate	40.67% (0.407)
Top-2 Category Match Rate	57.33% (0.573)
Top-3 Category Match Rate	60.67% (0.607)

Table 2. Metriche di qualità delle spiegazioni.

Graficamente abbiamo i seguenti risultati:

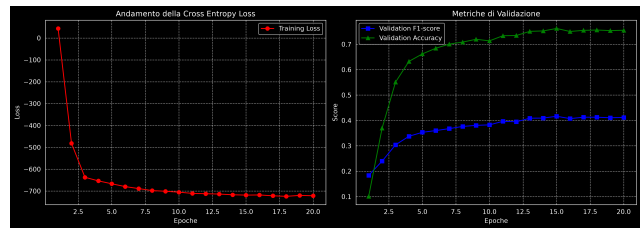


Figure 2. learning curves LR 0.0001 Win_3

Modello 2: Learning Rate = 0.0001, Window Size = 5

Questa configurazione mantiene una curva di apprendimento stabile e sovrapponibile al Modello precedente (fig:3). Anche le performance predittive restano pressoché inalterate, con una Recall di 0.839 e un F1-Score di 0.408 sulla classe *unfair*. Il vero punto di forza di questa con-

figurazione risiede nell'interpretabilità: registra il punteggio migliore in assoluto tra gli esperimenti, con un *Top-1 Match Rate* del 47.3% e un *Top-3 Match Rate* del 63.3%. Il modello dimostra un'eccellente capacità di allineare semanticamente le clausole individuate ai prototipi corretti.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.976	0.744	0.844	2045
unfair	0.269	0.839	0.408	230
Accuracy			0.753	2275
Macro Avg	0.623	0.791	0.626	2275
Weighted Avg	0.905	0.753	0.800	2275

Table 3. Classification Report per il modello LR_0.0001.Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	150
Top-1 Category Match Rate	47.33% (0.473)
Top-2 Category Match Rate	58.00% (0.580)
Top-3 Category Match Rate	63.33% (0.633)

Table 4. Metriche di qualità delle spiegazioni.

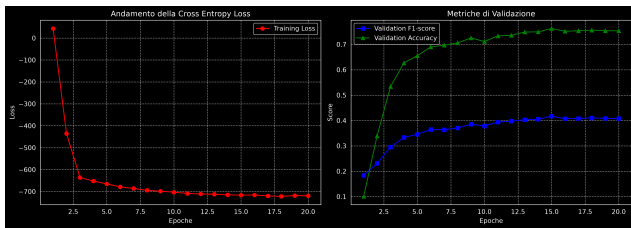


Figure 3. learning curves LR 0.0001 Win 5

Modello 3: Learning Rate = 0.0005, Window Size = 3

L'incremento del learning rate porta a una convergenza più rapida e a un drastico miglioramento delle performance di classificazione pura (fig:4): l'accuratezza globale sale a 0.835 e l'F1-Score per la classe *unfair* tocca il suo massimo (0.507), grazie a un netto miglioramento della Precision (0.363). Tuttavia, questo vantaggio predittivo causa un crollo nell'*Explanation Quality*: il *Top-1 Match Rate* precipita all'11.6% e il Top-3 si ferma al 25.2%. Il modello classifica bene, ma fornisce spiegazioni incoerenti.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.979	0.835	0.901	2045
unfair	0.363	0.839	0.507	230
Accuracy			0.835	2275
Macro Avg	0.671	0.837	0.704	2275
Weighted Avg	0.917	0.835	0.861	2275

Table 5. Classification Report per il modello LR_0.0005.Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	147
Top-1 Category Match Rate	11.56% (0.116)
Top-2 Category Match Rate	20.41% (0.204)
Top-3 Category Match Rate	25.17% (0.252)

Table 6. Metriche di qualità delle spiegazioni.

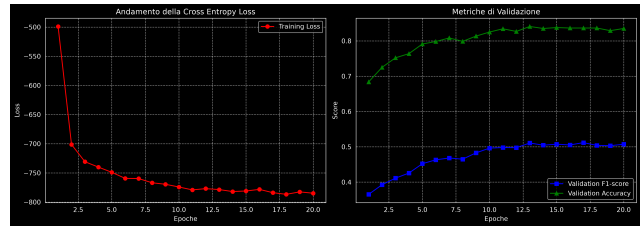


Figure 4. learning curves LR 0.0005 Win 3

Modello 4: Learning Rate = 0.0005, Window Size = 5

Le curve di apprendimento di questo modello (fig:5) mostrano l'innesco del meccanismo di *early stopping* all'epoca 18, sintomo di una mancata generalizzazione ulteriore. Le performance di classificazione rimangono alte (Accuracy 0.830, F1-Score *unfair* 0.490). Anche in questo caso, l'interpretabilità risulta severamente compromessa rispetto ai modelli con LR inferiore, registrando un *Top-1* del 14.5% e un *Top-3* del 32.4%.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.975	0.832	0.898	2045
unfair	0.352	0.809	0.490	230
Accuracy			0.830	2275
Macro Avg	0.663	0.820	0.694	2275
Weighted Avg	0.912	0.830	0.857	2275

Table 7. Classification Report per il modello LR_0.0005.Win_5.

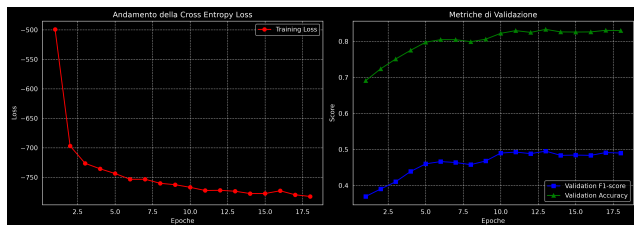


Figure 5. learning curves LR 0.0005 Win 5

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	145
Top-1 Category Match Rate	14.48% (0.145)
Top-2 Category Match Rate	23.45% (0.234)
Top-3 Category Match Rate	32.41% (0.324)

Table 8. Metriche di qualità delle spiegazioni.

2.2.1 Impatto del Numero di Prototipi

Per completezza, è stata condotta una seconda serie di esperimenti aumentando il numero di prototipi associati a ciascuna categoria da 3 a 10. Il numero totale di prototipi passa quindi da 24 a 80, mantenendo invariati gli altri iperparametri considerati.

L'obiettivo dell'esperimento è verificare se una maggiore quantità di prototipi consenta di rappresentare meglio la variabilità semantica interna alle singole categorie di clause unfair.

Modello con 80 prototipi, LR=0.0005, Window Size = 3 le prestazioni predittive restano elevate: *Accuracy* pari a 0.840, *Recall* sulla classe unfair pari a 0.809 e *F1-score* pari a 0.506. Dal punto di vista XAI, però, il Top-1 Category Match Rate è piuttosto basso, pari al 17.0%, mentre Top-2 e Top-3 salgono rispettivamente al 45.2% e 57.0%

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.975	0.844	0.905	2045
unfair	0.368	0.809	0.506	230
Accuracy			0.840	2275
Macro Avg	0.672	0.826	0.705	2275
Weighted Avg	0.914	0.840	0.865	2275

Table 9. Classification Report per il modello con 80 prototipi, LR_0.0005_Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	135
Top-1 Category Match Rate	17.04% (0.170)
Top-2 Category Match Rate	45.19% (0.452)
Top-3 Category Match Rate	57.04% (0.570)

Table 10. Metriche di qualità delle spiegazioni per il modello con 80 prototipi, LR_0.0005_Win_3.

Modello con 80 prototipi, LR=0.0005, Window Size = 5 la classificazione migliora leggermente: *Accuracy* 0.843 e *F1-score* unfair ancora pari a 0.506, con *Recall* 0.796. La differenza più interessante riguarda però l'interpretabilità: il Top-1 sale al 38.1%, il Top-2 al 56.8% e il Top-3 al 69.8%.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
fair	0.974	0.848	0.907	2045
unfair	0.371	0.796	0.506	230
Accuracy			0.843	2275
Macro Avg	0.672	0.822	0.706	2275
Weighted Avg	0.913	0.843	0.866	2275

Table 11. Classification Report per il modello con 80 prototipi, LR_0.0005_Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	139
Top-1 Category Match Rate	38.13% (0.381)
Top-2 Category Match Rate	56.83% (0.568)
Top-3 Category Match Rate	69.78% (0.698)

Table 12. Metriche di qualità delle spiegazioni per il modello con 80 prototipi, LR_0.0005_Win_5.

2.3. Analisi Comparativa e Trade-off

Il confronto tra le diverse configurazioni evidenzia come le prestazioni del modello dipendano dall'interazione tra tre principali iperparametri: il *learning rate*, la *window size* e il numero di prototipi utilizzati per rappresentare ciascuna categoria. I risultati mostrano inoltre un evidente trade-off tra performance predittiva e qualità delle spiegazioni.

- **L'impatto del Learning Rate:** considerando inizialmente la configurazione con 24 prototipi complessivi (3 per categoria), l'aumento del learning rate da 0.0001 a 0.0005 determina un netto miglioramento delle prestazioni predittive. L'*F1-score* della classe *unfair*

passa infatti da valori prossimi a 0.41 a valori prossimi a 0.50.

Tale miglioramento, tuttavia, è accompagnato da una riduzione della coerenza semantica delle spiegazioni. Con 24 prototipi e $LR = 0.0005$, il Top-1 Category Match Rate scende fino a valori inferiori al 15%, mentre con $LR = 0.0001$ raggiunge valori superiori al 40%.

Questo comportamento suggerisce che, a parità di numero di prototipi, un'ottimizzazione più aggressiva della funzione di costo possa favorire maggiormente l'obiettivo di classificazione, riducendo però l'allineamento tra i prototipi attivati e le categorie semantiche alle quali essi erano stati originariamente associati.

- **L'impatto della Window Size:** con 24 prototipi e $LR = 0.0001$, l'aumento della *window size* da 3 a 5 parole non produce variazioni significative nelle prestazioni di classificazione, mantenendo l'F1-score della classe *unfair* intorno a 0.41.

Al contrario, si osserva un miglioramento della qualità delle spiegazioni: il Top-1 Category Match Rate passa dal 40.7% al 47.3%, mentre il Top-3 aumenta dal 60.7% al 63.3%. Questo risultato suggerisce che una finestra leggermente più ampia possa fornire un contesto locale più informativo, favorendo una maggiore coerenza tra il frammento selezionato e il prototipo attivato.

Un comportamento analogo, ma ancora più marcato, emerge nella configurazione con 80 prototipi. A parità di $LR = 0.0005$, l'F1-score della classe *unfair* rimane invariato a 0.506 passando da una *window size* di 3 a 5, mentre il Top-1 Match Rate cresce dal 17.0% al 38.1% e il Top-3 dal 57.0% al 69.8%. La dimensione della finestra sembra quindi influenzare soprattutto la qualità delle spiegazioni, più che la capacità predittiva del modello.

- **L'impatto del numero di prototipi:** aumentando il numero di prototipi da 24 a 80 (da 3 a 10 prototipi per categoria), il modello dispone di una rappresentazione più granulare della variabilità semantica per le singole categorie.

Il confronto risulta particolarmente interessante per $LR = 0.0005$ e *window size*=5. Con 24 prototipi il modello raggiunge un F1-score sulla classe *unfair* pari a 0.490, con Top-1 e Top-3 Match Rate rispettivamente pari al 14.5% e al 32.4%. Utilizzando 80 prototipi, l'F1-score sale leggermente a 0.506, mentre il Top-1 raggiunge il 38.1% e il Top-3 il 69.8%.

Questi risultati suggeriscono che un numero maggiore di prototipi possa contribuire non soltanto a rappresentare meglio le diverse modalità con cui una categoria *unfair* può manifestarsi nel testo, ma anche a preservare maggiormente l'allineamento semantico delle spiegazioni durante l'ottimizzazione a discapito però dei tempi di addestramento che possono diventare proibitivi.

Conclusioni del Task Binario: Gli esperimenti mostrano che non esiste una singola configurazione ottimale rispetto a tutte le metriche considerate. Con un numero ridotto di prototipi, un learning rate basso favorisce maggiormente la qualità delle spiegazioni, mentre valori più elevati del learning rate migliorano le prestazioni di classificazione.

L'aumento del numero di prototipi modifica tuttavia questo trade-off. In particolare, la configurazione con 80 prototipi, $LR = 0.0005$ e *window size*=5 raggiunge un F1-score pari a 0.506 sulla classe *unfair*, mantenendo al tempo stesso un Top-3 Category Match Rate pari al 69.8%.

Rappresenta quindi il miglior compromesso osservato tra capacità predittiva e qualità delle spiegazioni tra le configurazioni con learning rate elevato.

La configurazione con 24 prototipi, $LR = 0.0001$ e *window size*=5 rimane invece quella con il più elevato Top-1 Category Match Rate (47.3%), evidenziando una migliore spiegazione, al fronte di prestazioni predittive inferiori.

3. Analisi dei risultati (Caso Multiclasse)

Dopo aver analizzato il problema nella sua formulazione binaria, distinguendo solamente tra clausole *fair* e *unfair*, è stata considerata una seconda configurazione con l'obiettivo di identificare direttamente la categoria di clausola sleale.

Il dataset *Unfair-ToS* mette a disposizione otto categorie di clausole *unfair* e consente, di associare più categorie alla stessa istanza. L'architettura utilizzata in questo lavoro è stata tuttavia adattata a una formulazione multiclasse a singola etichetta, per questioni implementative è risultata la più diretta.

Il problema viene quindi definito come una classificazione a nove classi:

$$\mathcal{Y} = \{0, 1, \dots, 7, 8\},$$

dove le classi da 0 a 7 corrispondono alle otto categorie *unfair* del dataset originale, mentre la classe 8 rappresenta le clausole *fair*.

3.1. Costruzione del Dataset Multiclasse

Nel dataset originale, una clausola può essere associata a nessuna, una o più categorie *unfair*. Per poter utilizzare

una formulazione multiclasse a singola etichetta, è stata introdotta una procedura di riduzione delle etichette.

Per ogni clausola priva di etichette unfair viene assegnata la classe 8, corrispondente alla categoria *fair*. Quando è presente una sola etichetta unfair, tale categoria viene utilizzata direttamente come target di classificazione. Nel caso in cui una clausola presenti più etichette contemporaneamente, viene invece selezionata la categoria con indice più basso:

$$y = \begin{cases} 8, & \text{se } Y = \emptyset, \\ \min(Y), & \text{altrimenti,} \end{cases}$$

dove Y rappresenta l'insieme delle categorie associate alla clausola nel dataset originale.

Ad esempio, una clausola annotata con le categorie $\{1, 6\}$ viene trasformata in un'istanza appartenente esclusivamente alla classe 1.

Per completezza riporto la tabella tab:13:

Classe	Occorrenze originali	Usate nel training	Etichette perse
cat_0	191	191	0
cat_1	139	130	9
cat_2	122	97	25
cat_3	73	43	30
cat_4	76	74	2
cat_5	39	39	0
cat_6	34	28	6
cat_7	28	28	0
fair	4902	4902	0

Table 13. Distribuzione delle classi e occorrenze nel training set.

La trasformazione da multilabel a multiclasse comporta inevitabilmente una perdita di informazione per le clausole associate a più categorie.

3.2. Gestione dello Sbilanciamento delle Classi

La formulazione multiclasse presenta un forte sbilanciamento tra le categorie. In particolare, la classe *fair* risulta molto più numerosa rispetto alle singole categorie unfair.

Per ridurre l'effetto di tale sbilanciamento durante l'addestramento, il DataLoader di training utilizza un `WeightedRandomSampler`. A ciascuna istanza viene assegnato un peso inversamente proporzionale alla radice quadrata della frequenza della relativa classe:

$$w_i = \frac{1}{\sqrt{N_{y_i}}},$$

dove N_{y_i} rappresenta il numero di esempi appartenenti alla classe dell'istanza i .

In questo modo, gli esempi appartenenti alle categorie meno frequenti hanno una probabilità maggiore

di essere selezionati durante la costruzione dei batch. Il campionamento viene effettuato con reinserimento (`replacement=True`), pertanto una stessa istanza può essere selezionata più volte nel corso di un'epoca.

La scelta della radice quadrata permette di attenuare lo sbilanciamento senza imporre un bilanciamento uniforme tra le classi.

3.3. Costruzione dei Prototipi

La procedura di costruzione dei prototipi mantiene l'approccio supervisionato già visto nel caso binario, ma viene esteso alla formulazione a nove classi.

Per ciascuna delle otto categorie unfair vengono selezionate le clausole del training set associate alla categoria considerata. Le clausole vengono rappresentate mediante Sentence-Transformer e successivamente suddivise tramite K-Means, utilizzando un numero prefissato di cluster per categoria. Il centroide di ciascun cluster costituisce l'inizializzazione del relativo prototipo.

In questo caso a differenza della configurazione binaria precedente, nella formulazione multiclasse vengono introdotti anche prototipi dedicati esplicitamente alla classe *fair*.

Una clausola viene considerata *fair* quando nessuna delle otto etichette *unfair* è presente. Tali clausole vengono raccolte separatamente e clusterizzate tramite K-Means, generando un insieme specifico di prototipi rappresentativi della classe *fair*.

Il numero complessivo di prototipi è quindi dato da

$$P = 8P_u + P_f,$$

dove P_u rappresenta il numero di prototipi assegnati a ciascuna categoria unfair e P_f il numero di prototipi dedicati alla classe *fair*.

3.4. Adattamento del Modello alla Classificazione Multiclasse

L'architettura interna di ProtoLens rimane sostanzialmente invariata rispetto al caso binario. Il testo viene confrontato con i prototipi e, per ciascuno di essi, viene calcolato un valore di similarità che rappresenta il grado di attivazione del prototipo sull'input.

La principale modifica riguarda il livello di classificazione finale. Il layer completamente connesso riceve le attivazioni dei prototipi e produce nove logits, uno per ciascuna delle classi considerate:

$$\mathbf{o} = [o_0, o_1, \dots, o_8].$$

Durante l'addestramento, tali logits vengono confrontati con la classe target mediante la funzione di Cross-Entropy, contribuendo al calcolo della funzione di costo utilizzata per aggiornare i parametri del modello.

Durante la fase di valutazione e inferenza, invece, la classe predetta viene ottenuta selezionando il logit di valore massimo:

$$\hat{y} = \arg \max_c o_c.$$

3.5. Risultati con 3 Prototipi per Categoria Unfair

In una prima configurazione sperimentale sono stati utilizzati 3 prototipi per ciascuna delle otto categorie unfair e 15 prototipi dedicati alla classe fair, per un totale di

$$P = 8 \cdot 3 + 15 = 39$$

prototipi.

Poiché la classe *fair* risulta significativamente più numerosa rispetto alle singole categorie unfair, è stato utilizzato un numero maggiore di prototipi per rappresentarne la maggiore variabilità interna.

La scelta di utilizzare un numero maggiore di prototipi per la classe fair è di **natura euristica** e mira a evitare che una classe molto ampia (probabilmente molto eterogenea) venga rappresentata da un numero eccessivamente ridotto di centroidi.

Sono state confrontate quattro configurazioni, ottenute combinando due valori di learning rate (0.0001 e 0.0005) e due valori della window size (3 e 5).

I risultati evidenziano innanzitutto un forte effetto del learning rate. Con $LR = 0.0001$ il modello mantiene prestazioni molto elevate sulla classe fair, ma mostra difficoltà nel riconoscimento delle categorie unfair meno rappresentate. In entrambe le configurazioni con learning rate basso, alcune categorie presentano infatti un F1-score nullo, con un Macro F1 complessivo prossimo a 0.30.

Aumentando il learning rate a 0.0005, le prestazioni sulle classi minoritarie migliorano sensibilmente. Con una window size pari a 3 il Macro F1 raggiunge 0.574, mentre con una finestra di 5 parole sale a 0.612. Quest'ultima rappresenta quindi la configurazione con le migliori prestazioni predittive tra quelle considerate.

Learning Rate = 0.0001, Window Size = 3

La prima configurazione utilizza un learning rate pari a 0.0001 e una window size di 3 parole. Il modello mostra buone prestazioni complessive sulla classe *fair*, mentre incontra maggiori difficoltà nel riconoscimento delle categorie unfair meno frequenti. Il Macro F1-Score risulta pari a 0.309 e alcune classi minoritarie non vengono riconosciute correttamente.

Dal punto di vista dell'interpretabilità, il modello classifica correttamente 44 istanze unfair su 172. Su questo sottoinsieme, il Top-1 Category Match Rate è pari al 27.3%, mentre il Top-3 raggiunge il 65.9%.

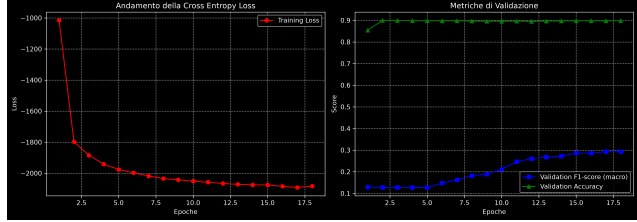


Figure 6. learning curves LR 0.0001 window size = 3

Classe / Metrica	Precision	Recall	F1-Score	Supporto
cat_0	0.591	0.342	0.433	38
cat_1	0.583	0.368	0.452	38
cat_2	0.529	0.237	0.327	38
cat_3	0.000	0.000	0.000	13
cat_4	0.000	0.000	0.000	23
cat_5	0.333	0.077	0.125	13
cat_6	0.450	0.562	0.500	16
cat_7	0.000	0.000	0.000	7
fair	0.919	0.974	0.946	1435
Micro Avg	0.899	0.891	0.895	1621
Macro Avg	0.378	0.285	0.309	1621
Weighted Avg	0.861	0.891	0.872	1621
Samples Avg	0.899	0.896	0.897	1621

Table 14. Classification Report per il modello LR-0.0001-Win-3.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	44
Top-1 Category Match Rate	27.27% (0.273)
Top-2 Category Match Rate	45.45% (0.455)
Top-3 Category Match Rate	65.91% (0.659)

Table 15. Metriche di qualità delle spiegazioni per il modello LR-0.0001-Win-3.

Learning Rate=0.0001, Window Size=5. La seconda configurazione mantiene invariato il learning rate ($LR = 0.0001$) e aumenta la window size da 3 a 5 parole. Le prestazioni predittive rimangono sostanzialmente simili alla configurazione precedente, con un Macro F1-Score pari a 0.305. Anche in questo caso alcune categorie unfair minoritarie non vengono correttamente riconosciute.

La qualità delle spiegazioni risulta invece leggermente superiore: su 40 istanze unfair correttamente classificate, il Top-1 Category Match Rate raggiunge il 35.0%, mentre il Top-3 arriva al 72.5%.

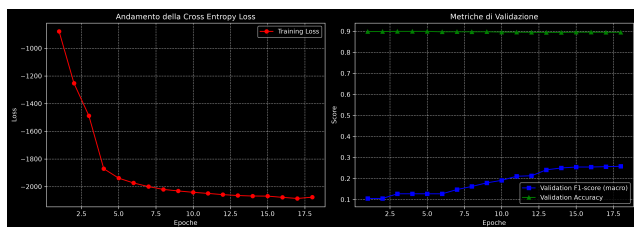


Figure 7. learning curves LR 0.0001 window size = 5

Classe / Metrica	Precision	Recall	F1-Score	Supporto
cat_0	0.591	0.342	0.433	38
cat_1	0.619	0.342	0.441	38
cat_2	0.667	0.211	0.320	38
cat_3	0.000	0.000	0.000	13
cat_4	0.000	0.000	0.000	23
cat_5	0.400	0.154	0.222	13
cat_6	0.400	0.375	0.387	16
cat_7	0.000	0.000	0.000	7
fair	0.915	0.977	0.945	1435
Micro Avg	0.899	0.891	0.895	1621
Macro Avg	0.399	0.267	0.305	1621
Weighted Avg	0.861	0.891	0.870	1621
Samples Avg	0.899	0.896	0.897	1621

Table 16. Classification Report per il modello LR_0.0001.Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	40
Top-1 Category Match Rate	35.00% (0.350)
Top-2 Category Match Rate	55.00% (0.550)
Top-3 Category Match Rate	72.50% (0.725)

Table 17. Metriche di qualità delle spiegazioni per il modello LR_0.0001.Win_5.

Learning Rate=0.0005, Window Size=3. La terza configurazione utilizza un learning rate più elevato, pari a 0.0005, mantenendo una window size di 3 parole. Rispetto ai modelli con learning rate inferiore, si osserva un miglioramento nella capacità di distinguere le categorie unfair. Il Macro F1-Score raggiunge infatti 0.574 e tutte le categorie presentano un F1-Score maggiore di zero.

Il modello classifica correttamente 111 istanze unfair su 172. Tra queste, il Top-1 Category Match Rate è pari al 32.4%, il Top-2 al 45.0% e il Top-3 al 58.6%.

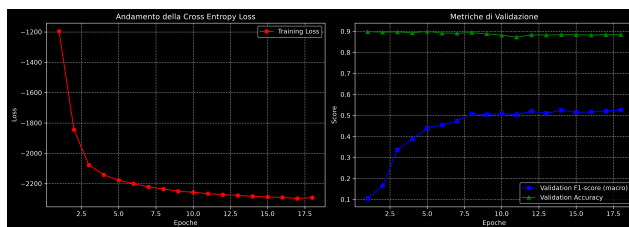


Figure 8. learning curves LR 0.0005 window size = 3

Classe / Metrica	Precision	Recall	F1-Score	Supporto
cat_0	0.538	0.737	0.622	38
cat_1	0.491	0.711	0.581	38
cat_2	0.458	0.579	0.512	38
cat_3	0.571	0.308	0.400	13
cat_4	0.625	0.435	0.513	23
cat_5	0.667	0.462	0.545	13
cat_6	0.542	0.812	0.650	16
cat_7	0.375	0.429	0.400	7
fair	0.962	0.930	0.946	1435
Micro Avg	0.901	0.893	0.897	1621
Macro Avg	0.581	0.600	0.574	1621
Weighted Avg	0.912	0.893	0.900	1621
Samples Avg	0.901	0.898	0.899	1621

Table 18. Classification Report per il modello LR_0.0005.Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	111
Top-1 Category Match Rate	32.43% (0.324)
Top-2 Category Match Rate	45.05% (0.450)
Top-3 Category Match Rate	58.56% (0.586)

Table 19. Metriche di qualità delle spiegazioni per il modello LR_0.0005.Win_3.

Learning Rate=0.0005, Window Size=5. La quarta configurazione combina un learning rate pari a 0.0005 con una window size di 5 parole. Tra i quattro modelli considerati in questa prima serie di esperimenti, questa configurazione ottiene le migliori prestazioni predittive complessive, con un Macro F1-Score pari a 0.612.

Il modello classifica correttamente 112 istanze unfair su 172. Per quanto riguarda la qualità delle spiegazioni, il Top-1 Category Match Rate è pari al 29.5%, mentre il Top-3 raggiunge il 47.3%. Nonostante le migliori prestazioni di classificazione, la coerenza delle spiegazioni risulta quindi inferiore rispetto ad alcune delle configurazioni con learning rate più basso.

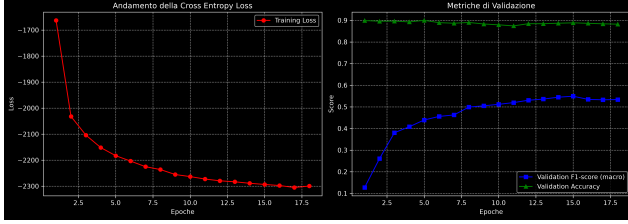


Figure 9. learning curves LR 0.0005 window size = 5

Classe / Metrica	Precision	Recall	F1-Score	Supporto
cat_0	0.500	0.684	0.578	38
cat_1	0.510	0.684	0.584	38
cat_2	0.431	0.579	0.494	38
cat_3	0.667	0.462	0.545	13
cat_4	0.688	0.478	0.564	23
cat_5	0.667	0.462	0.545	13
cat_6	0.560	0.875	0.683	16
cat_7	0.571	0.571	0.571	7
fair	0.963	0.930	0.946	1435
Micro Avg	0.902	0.895	0.898	1621
Macro Avg	0.617	0.636	0.612	1621
Weighted Avg	0.914	0.895	0.902	1621
Samples Avg	0.902	0.899	0.900	1621

Table 20. Classification Report per il modello LR_0.0005_Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True unfair (n_{unfair})	172
Correctly classified unfair	112
Top-1 Category Match Rate	29.46% (0.295)
Top-2 Category Match Rate	40.18% (0.402)
Top-3 Category Match Rate	47.32% (0.473)

Table 21. Metriche di qualità delle spiegazioni per il modello LR_0.0005_Win_5.

3.6. Risultati con 10 Prototipi per Categoria Unfair

In una seconda configurazione sperimentale, il numero di prototipi associati a ciascuna categoria unfair è stato aumentato da 3 a 10, mantenendo 15 prototipi dedicati alla classe fair.

Il numero totale di prototipi passa quindi da 39 a

$$P = 8 \cdot 10 + 15 = 95.$$

L'obiettivo di questo esperimento è verificare se una maggiore capacità prototipale consenta di rappresentare in modo più accurato la variabilità semantica interna alle categorie unfair.

Prototipi	LR	Window	Macro F1	Unfair corretti	Top-1	Top-3
39	0.0005	3	0.574	111	32.4%	58.6%
39	0.0005	5	0.612	112	29.5%	47.3%
95	0.0005	3	0.665	128	39.1%	76.6%
95	0.0005	5	0.647	124	25.0%	58.9%

Table 22. Confronto delle prestazioni al variare del numero di prototipi e della dimensione della window.

L'aumento del numero di prototipi produce un miglioramento evidente sia nelle prestazioni predittive sia nella qualità delle spiegazioni. In particolare, con $LR = 0.0005$ e $window\ size=3$, il Macro F1-Score cresce da 0.574 a 0.665, mentre il numero di istanze unfair correttamente classificate passa da 111 a 128.

Anche la qualità delle spiegazioni migliora in modo significativo: il Top-1 Category Match Rate aumenta dal 32.4% al 39.1%, mentre il Top-3 passa dal 58.6% al 76.6%.

Questi risultati suggeriscono che una maggiore quantità di prototipi consenta di rappresentare meglio la variabilità semantica delle categorie unfair, migliorando contemporaneamente la capacità predittiva e la qualità delle spiegazioni.

3.7. Analisi Complessiva del Task Multiclasse

Nel complesso, i risultati mostrano che il learning rate e il numero di prototipi incidono maggiormente sulle prestazioni rispetto alla window size. Le configurazioni con $LR = 0.0001$ mostrano maggiori difficoltà nel riconoscimento delle classi unfair minoritarie, mentre con $LR = 0.0005$ il Macro F1-Score migliora sensibilmente.

L'aumento dei prototipi da 39 a 95 porta inoltre a un ulteriore miglioramento sia nella classificazione sia nella qualità delle spiegazioni. In particolare, la configurazione con $LR = 0.0005$, $window\ size=3$, 10 prototipi per ciascuna categoria unfair e 15 prototipi per la classe fair ottiene il miglior compromesso complessivo, con Macro F1-Score pari a 0.665 e Top-3 Category Match Rate pari al 76.6%.

I risultati suggeriscono quindi che una più prototipi consentano di rappresentare meglio la variabilità semantica delle categorie unfair, migliorando anche l'accuratezza delle spiegazioni.

4. Classificatori Binari Specializzati

Come terza configurazione sperimentale è stato adottato un approccio basato su classificatori binari specializzati. A differenza del modello OR, che effettua una singola decisione binaria tra clausole fair e unfair, e del modello multiclasse, in questa configurazione viene addestrato un classificatore indipendente per una specifica categoria di clausola unfair.

Sono state selezionate le tre categorie maggiormente rappresentate nel dataset:

- *Limitation of Liability*;
- *Unilateral Termination*;
- *Unilateral Change*.

Per ciascuna categoria target c viene costruito un dataset binario nel quale la label è definita come

$$y_c = \begin{cases} 1, & \text{se la categoria } c \text{ è presente nella clausola,} \\ 0, & \text{altrimenti.} \end{cases}$$

La classe negativa comprende quindi sia le clausole fair sia le clausole unfair appartenenti a categorie differenti da quella target. Ciascun classificatore risponde pertanto alla domanda: “la specifica tipologia di clausola unfair è presente nel testo?”.

Questo approccio consente inoltre di preservare la natura originariamente multi-label del dataset: una clausola contenente più categorie può essere considerata positiva da più classificatori specializzati indipendenti.

4.1. Prima Implementazione

In una prima implementazione dei classificatori specializzati è stata mantenuta lo stesso set di prototipi utilizzato negli esperimenti precedenti. In particolare, i prototipi venivano costruiti rispetto alle otto categorie unfair originali, alle quali veniva aggiunto un insieme di prototipi rappresentativi delle clausole fair (15 prototipi).

La configurazione utilizzata prevedeva quindi prototipi appartenenti alle categorie differenti, indipendentemente dalla categoria specifica riconosciuta dal classificatore binario (forse questo aspetto creava varie problematiche).

Ad esempio, nel classificatore specializzato per *Limitation of Liability*, il modello disponeva non soltanto di prototipi appartenenti alla categoria *Limitation*, ma anche di prototipi relativi a *Termination*, *Change* e alle altre categorie unfair.

Dal punto di vista predittivo credo che questa configurazione rimanga valida. Tuttavia, porta a una criticità dal punto di vista dell’**interpretabilità**: non esiste infatti alcun vincolo che permetta a una predizione positiva per una determinata categoria a essere supportata principalmente dai prototipi appartenenti alla stessa categoria.

Quello che accadeva è che nel caso del classificatore *Limitation of Liability* i prototipi utilizzati per la spiegazione erano di classi sbagliate (stava imparando a classificare bene per i motivi sbagliati).

Table 23. Risultati - Limitation

LR	Win	Test	T. True	T. Corr.	Top 1	Top 2	Top 3
0.0001	3	1607	38	25	0.0000	0.0400	0.0400
0.0001	5	1607	38	24	0.0000	0.0833	0.0833
0.0005	3	1607	38	26	0.0000	0.0769	0.0769
0.0005	5	1607	38	27	0.0000	0.0000	0.0370

Table 24. Risultati - Unilateral Termination

LR	Win	Test	T. True	T. Corr.	Top 1	Top 2	Top 3
0.0001	3	1607	38	0	NaN	NaN	NaN
0.0001	5	1607	38	27	0.8889	0.9630	1.0000
0.0005	3	1607	38	32	0.6563	0.7813	0.8438
0.0005	5	1607	38	31	0.0000	0.0000	0.0000

Table 25. Risultati - Unilateral Change

LR	Win	Test	T. True	T. Corr.	Top 1	Top 2	Top 3
0.0001	3	1607	38	26	0.5000	0.6154	0.6538
0.0001	5	1607	38	24	0.5833	0.6667	0.7917
0.0005	3	1607	38	25	0.6400	0.7600	0.8000
0.0005	5	1607	38	25	0.6000	0.6800	0.8800

4.2. Seconda Implementazione: Prototipi Specifici per il Task Binario

Per rendere coerente la rappresentazione prototipale con il task specializzato, è stata introdotta una seconda configurazione nella quale anche la costruzione dei prototipi segue direttamente la struttura del problema binario.

Per il classificatore dedicato a *Limitation of Liability*, le due classi utilizzate per il clustering sono:

$$P^+ = \{\text{clausole Limitation of Liability}\},$$

$$P^- = \{\text{tutte le clausole non-Limitation}\}.$$

Vengono quindi eseguite due procedure di clustering indipendenti, una sulla classe positiva e una sulla classe negativa. I centroidi ottenuti costituiscono i prototipi iniziali utilizzati da ProtoLens.

Nella configurazione attualmente analizzata vengono utilizzati:

- 10 prototipi per la classe *Limitation of Liability*;
- 20 prototipi per la classe *Rest*.

Il numero maggiore di prototipi assegnato alla classe *Rest* è motivato dalla maggiore eterogeneità della classe negativa, che comprende sia clausole **fair** sia clausole appartenenti alle altre categorie **unfair**.

Il numero complessivo di prototipi è pertanto pari a

$$P = 10 + 20 = 30.$$

4.3. Valutazione Predittiva e Interpretabilità

La valutazione predittiva considera le due classi:

- *Rest*;
- *Limitation of Liability*.

Per analizzare il contributo dei prototipi alla decisione finale viene utilizzata la procedura di spiegazione già introdotta negli esperimenti precedenti. Per la classe predetta $\hat{y}$, il contributo del prototipo p è definito come

$$C_p = s_p w_{\hat{y},p},$$

dove s_p rappresenta la similarità tra la rappresentazione del testo e il prototipo p , mentre $w_{\hat{y},p}$ è il peso del layer lineare associato a tale prototipo per la classe predetta.

I prototipi vengono ordinati in ordine decrescente rispetto a C_p e vengono estratti i primi K prototipi maggiormente influenti.

4.3.1 Top-K Prototype Match Rate

Nel nuovo setting binario anche i metadati dei prototipi sono definiti direttamente rispetto alle due classi:

$$\text{prototype_class} = \begin{cases} 0, & \text{Rest,} \\ 1, & \text{Limitation of Liability.} \end{cases}$$

La qualità della spiegazione viene valutata sulle clausole realmente appartenenti alla categoria target e correttamente classificate come positive.

Per ciascuna di tali clausole vengono calcolate le seguenti metriche:

- **Top-1 Match:** il prototipo con contributo maggiore appartiene alla classe positiva;
- **Top-2 Match:** almeno uno dei primi due prototipi appartiene alla classe positiva;
- **Top-3 Match:** almeno uno dei primi tre prototipi appartiene alla classe positiva.

4.4. Analisi dei risultati

Modello 1: Learning Rate = 0.0001, Window Size = 3

Le curve di apprendimento del modello (fig:10) mostrano una progressiva riduzione della training loss e un incremento dell'F1-Score di validazione fino alle ultime epoche. Sul test set, il modello raggiunge un'Accuracy pari a 0.918 e un F1-Score sulla classe *Limitation* pari a 0.328. Il Recall della classe target risulta elevato (0.842), mentre la Precision rimane contenuta (0.204), indicando la presenza di numerosi falsi positivi. Dal punto di vista dell'interpretabilità,

il modello ottiene un *Top-1* del 31.25%, un *Top-2* del 50.0% e un *Top-3* del 59.38%.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
Rest	0.996	0.920	0.957	1569
Limitation	0.204	0.842	0.328	38
Accuracy			0.918	1607
Macro Avg	0.600	0.881	0.642	1607
Weighted Avg	0.977	0.918	0.942	1607

Table 26. Classification Report per il modello LR_0.0001_Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True Limitation ($n_{\text{Limitation}}$)	38
Correctly classified Limitation	32
Top-1 Limitation Match Rate	31.25% (0.313)
Top-2 Limitation Match Rate	50.00% (0.500)
Top-3 Limitation Match Rate	59.38% (0.594)

Table 27. Metriche di qualità delle spiegazioni per il modello LR_0.0001_Win_3.

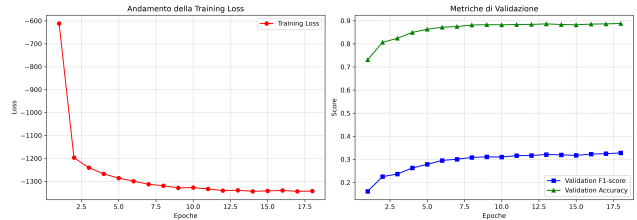


Figure 10. Learning curves del modello con LR 0.0001 e Window Size 3.

Modello 2: Learning Rate = 0.0001, Window Size = 5

Le curve di apprendimento del modello (fig:11) mostrano un andamento simile alla configurazione precedente, con una riduzione progressiva della training loss e una stabilizzazione delle metriche di validazione nelle ultime epoche. Sul test set, il modello raggiunge un'Accuracy pari a 0.912 e un F1-Score sulla classe *Limitation* pari a 0.312. Il Recall rimane pari a 0.842, mentre la Precision scende leggermente a 0.192. Dal punto di vista dell'interpretabilità, il *Top-1* risulta relativamente basso, pari al 21.88%, mentre il *Top-2* e il *Top-3* crescono rispettivamente fino all'84.38% e al 93.75%, mostrando che i prototipi associati alla classe target compaiono frequentemente tra le prime posizioni della spiegazione anche quando non occupano la prima posizione.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
Rest	0.996	0.914	0.953	1569
Limitation	0.192	0.842	0.312	38
Accuracy			0.912	1607
Macro Avg	0.594	0.878	0.633	1607
Weighted Avg	0.977	0.912	0.938	1607

Table 28. Classification Report per il modello LR_0.0001_Win_5.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
Rest	0.995	0.959	0.976	1569
Limitation	0.316	0.789	0.451	38
Accuracy			0.955	1607
Macro Avg	0.655	0.874	0.714	1607
Weighted Avg	0.979	0.955	0.964	1607

Table 30. Classification Report per il modello LR_0.0005_Win_3.

Metrica	Valore
Total test samples (n_{test})	1607
True Limitation ($n_{\text{Limitation}}$)	38
Correctly classified Limitation	32
Top-1 Limitation Match Rate	21.88% (0.219)
Top-2 Limitation Match Rate	84.38% (0.844)
Top-3 Limitation Match Rate	93.75% (0.938)

Table 29. Metriche di qualità delle spiegazioni per il modello LR_0.0001_Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True Limitation ($n_{\text{Limitation}}$)	38
Correctly classified Limitation	30
Top-1 Limitation Match Rate	3.33% (0.033)
Top-2 Limitation Match Rate	13.33% (0.133)
Top-3 Limitation Match Rate	56.67% (0.567)

Table 31. Metriche di qualità delle spiegazioni per il modello LR_0.0005_Win_3.

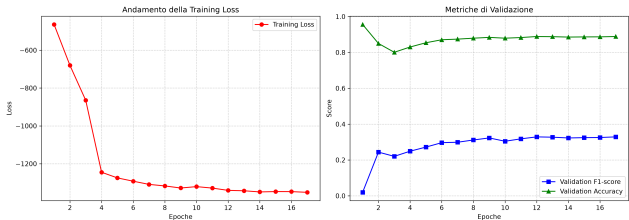


Figure 11. Learning curves del modello con LR 0.0001 e Window Size 5.

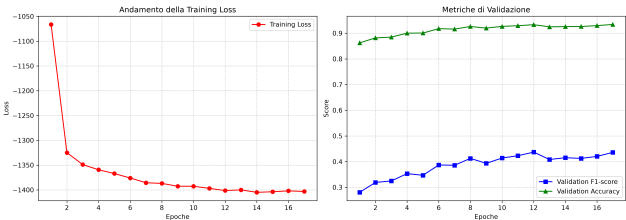


Figure 12. Learning curves del modello con LR 0.0005 e Window Size 3.

Modello 3: Learning Rate = 0.0005, Window Size = 3
 Le curve di apprendimento del modello (fig:12) mostrano una rapida riduzione della training loss nelle prime epoche e una successiva stabilizzazione. Sul test set, il modello raggiunge un'Accuracy pari a 0.955 e un F1-Score sulla classe *Limitation* pari a 0.451, con Precision pari a 0.316 e Recall pari a 0.789. Rispetto alle configurazioni con learning rate inferiore, si osserva quindi un miglior bilanciamento tra Precision e Recall. L'interpretabilità risulta invece più debole: il *Top-1* è pari al 3.33%, il *Top-2* al 13.33% e il *Top-3* al 56.67%, evidenziando come un miglioramento predittivo non corrisponda necessariamente a un miglioramento della coerenza delle spiegazioni.

Modello 4: Learning Rate = 0.0005, Window Size = 5
 Le curve di apprendimento del modello (fig:13) mostrano una riduzione rapida della training loss e una successiva stabilizzazione delle metriche di validazione. Sul test set, il modello raggiunge un'Accuracy pari a 0.955 e il miglior F1-Score sulla classe *Limitation*, pari a 0.463, con Precision 0.323 e Recall 0.816. Questa configurazione rappresenta quindi il miglior compromesso predittivo tra quelle analizzate. Anche l'interpretabilità risulta particolarmente solida: il *Top-1* raggiunge il 61.29%, il *Top-2* l'80.65% e il *Top-3* il 90.32%. In oltre il 90% delle clausole *Limitation* correttamente classificate, almeno uno dei primi tre prototipi più influenti appartiene quindi alla classe target.

Classe / Metrica	Precision	Recall	F1-Score	Supporto
Rest	0.995	0.959	0.977	1569
Limitation	0.323	0.816	0.463	38
Accuracy			0.955	1607
Macro Avg	0.659	0.887	0.720	1607
Weighted Avg	0.979	0.955	0.964	1607

Table 32. Classification Report per il modello LR_0.0005_Win_5.

Metrica	Valore
Total test samples (n_{test})	1607
True Limitation ($n_{\text{Limitation}}$)	38
Correctly classified Limitation	31
Top-1 Limitation Match Rate	61.29% (0.613)
Top-2 Limitation Match Rate	80.65% (0.806)
Top-3 Limitation Match Rate	90.32% (0.903)

Table 33. Metriche di qualità delle spiegazioni per il modello LR_0.0005_Win_5.

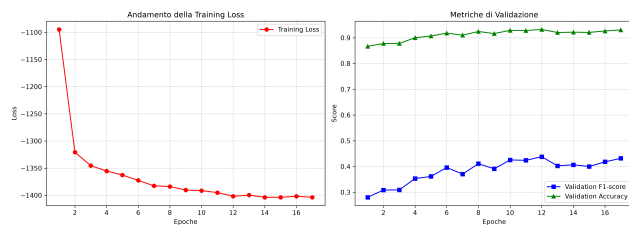


Figure 13. Learning curves del modello con LR 0.0005 e Window Size 5.

Il modello con **learning rate = 0.0005** e **window size = 5** risulta il migliore perché combina le migliori prestazioni predittive con la maggiore qualità delle spiegazioni: ottiene il miglior F1-Score sulla classe *Limitation* e valori Top-1, Top-2 e Top-3 superiori rispetto ad alcune configurazioni, risultando quindi il più equilibrato.

4.5. Confronto con le Configurazioni Precedenti

Il confronto tra il nuovo classificatore specializzato e le configurazioni precedenti evidenzia una chiara differenza tra prestazioni predittive e interpretabilità.

Modello	F1	Precision	Recall	Top-1	Top-2	Top-3
Nuovo specialista	0.463	0.323	0.816	0.613	0.806	0.903
Multiclasse (cat_0)	0.578	0.500	0.684	0.295*	0.402*	0.473*
Vecchio specialista binario	0.607	0.529	0.711	0.0	0.0	0.0

Table 34. Confronto tra il nuovo classificatore specializzato e le configurazioni precedenti.

* Attenzione: i Top-K del modello multiclasse sono calcolati sull'insieme delle 172 clausole unfair correttamente gestite, non specificamente sulle sole cat_0, quindi non sono perfettamente confrontabili con i Top-K dei due specialisti. Dal punto di vista predittivo, il precedente classificatore binario (prima implementazione) specializzato ottiene il miglior F1-score sulla classe target (0.607), seguito dal modello multiclasse sulla categoria 0 (0.578). Il nuovo classificatore specializzato raggiunge invece un F1-score pari a 0.463, ma presenta il Recall più elevato (0.816).

Dal punto di vista dell'interpretabilità, il nuovo modello probabilmente risulta superiore: il Top-1 raggiunge il 61.3%, il Top-2 l'80.6% e il Top-3 il 90.3%. La nuova costruzione dei prototipi, probabilmente comporta prestazioni predittive inferiori rispetto al vecchio specialista, ma spiegazioni molto più coerenti con la classe.